

LLM: 사전학습, 프롬프팅, 정렬

언어모델링: 다음 토큰 예측과 사전학습 · 파인튜닝 · 프롬프팅 · 포스트트레이닝: SFT, RLHF, DPO

Machine Learning 1 · 17주차

한국과학영재학교 (KSA)

Chapter 17

이번 주차 개요

- 의대생은 해부학·생리학·약리학 같은 **방대한 기초**를 먼저 익히고 나서야 특정 전공을 집중 훈련한다. LLM도 비슷: 방대한 텍스트로 다음 단어 예측을 익혀(**사전학습**) 폭넓은 지식을 흡수하고, 적은 데이터로 다듬고(**파인튜닝**), “사람이 실제로 원하는 방향”으로 정교하게 다듬는다 (**포스트트레이닝**).
- **17.1 언어모델링** — “다음 토큰 예측”이 **문장 전체 분포 학습과 동일한 과제**임을 사슬 분해로 보이고, BPE·softmax·온도·교차 엔트로피·PPL을 작은 예제로 확인한 뒤 사전학습/파인튜닝의 현대 도식으로 마무리.
- **17.2 프롬프팅** — 가중치 없이 **입력만으로** 행동을 바꾸기: zero-shot → few-shot → CoT 스펙트럼, temperature·top-k·top-p, 그리고 환각.
- **17.3 포스트트레이닝** — InstructGPT의 관찰에서 출발해 SFT → RLHF(보상모델 + PPO) → DPO를 짚고, LoRA/PEFT와 RAG·에이전트까지 모은 전체 지도.

이 장을 마치면 (1~16장 이후의 심화·응용 부록)

- softmax·교차 엔트로피·퍼플렉시티를 작은 예제로 직접 계산, BPE가 “단어 조각” 어휘를 만드는 이유를 설명할 수 있다.
- SFT·RLHF·DPO를 그림으로 구분하고, “그럴듯한 텍스트”가 “사람이 원하는 답변”과 왜 다르지 설명할 수 있다.

17.1 언어모델링: 다음 토큰 예측과 사전학습·파인튜닝

언어모델: “다음 토큰을 예측하는 함수”

- LLM은 텍스트를 **토큰(token)**의 시퀀스로 다룬다 — 단어 전체이거나, **더 흔하게** 단어의 일부 조각.
- 사전학습의 목표: 지금까지의 토큰이 주어졌을 때, 다음 토큰의 확률분포를 예측 —

$$P(x_t \mid x_1, x_2, \dots, x_{t-1})$$

- “프랑스의 수도는 ____”를 잘 완성하려면 모델은 “파리”라는 사실을 어딘가에 담아둬야 한다.
- 이런 문장이 수십억 개 쌓이면, 단순한 “다음 단어 맞추기”를 잘 하려는 모델이 **문법·사실 지식·추론 능력**까지 자연스럽게 습득.
- 확률은 **decoder-only Transformer**로 계산 — 미래 토큰을 보지 못하게 **마스킹**한 Ch. 12의 트랜스포머. “표현하는 장치” → “확률분포를 계산하는 실제 엔진”.
- 학습: 실제 다음 토큰과 예측 분포 사이의 **교차 엔트로피 손실**(Ch. 2.3 정보이론에서 나온 것과 같은 형태) 최소화.
- 토큰 단위 평균 $-\log p$ 에 지수 e 를 취하면 **퍼플렉시티**($\text{PPL} = e^{\mathcal{L}}$) — “다음 토큰을 고를 때 평균 몇 개의 선택지를 두고 헤매는가”. 낮을수록 확신 있는 예측.

사슬 분해: 다음 토큰 예측 = 문장 전체 분포 학습

- 어떤 분포 $P(x_1, \dots, x_n)$ 가 “이 문장만큼 그럴듯한 문장을 만들어내는 법”을 압축해 갖고 있다면 — 그것이 **언어모델**이다.
- 조건부 확률의 **사슬 분해**:

$$P(x_1, \dots, x_n) = \prod_{t=1}^n P(x_t \mid x_1, \dots, x_{t-1})$$

- “문장 전체의 확률”을 **한 토큰씩 예측하는 확률의 곱**으로 부수게 된다.
- 각 단계의 “다음 토큰 분포”를 잘 만들면 그 곱이 문장의 합리성 척도. 이 곱을 최대화하는 방향으로 학습하는 것이 곧 문장 전체를 잘 만드는 법을 배우는 것.

“다음 토큰 예측”은 사소한 게임이 아니라, 문장 전체의 분포를 학습하는 것과 정확히 동일한 과제.

Ch. 11에서 RNN으로 시퀀스를 모델링할 때 시간 단계의 예측을 곱해서 문장 확률을 계산했던 것과 같은 수학.

토큰은 왜 “단어 조각”인가: BPE

- 문법상 단어는 유한하지만 실제 쓰이는 단어는 **거의 무한** — 합성어, 고유명사, 새로 지어지는 말. 반면 벡터를 표현할 수 있는 수(= 어휘 사전 크기)는 **유한**: 실제 모델 대략 3만~20만.
- **BPE**(Byte Pair Encoding)가 이 모순의 표준 해결책:
 - 1 토큰을 문자(한글 글자, 바이트) 단위로 쪼갬.
 - 2 “문서에서 가장 자주 함께 나타나는 두 인접 토큰”을 **반복 병합(merge)**.
 - 3 어휘 사전이 목표 크기(예: 32,768개)가 될 때까지.
- 병합된 조각도 사전에 한 단위로 추가 ⇒ 잦은 조각(“은”, “의”, “하다”)은 한 토큰, 드문 단어는 여러 조각 — 자연스러운 계층.
- **보상**: 사전에 없던 전혀 새로운 단어도 **아는 조각의 조합**으로 분해 → **OOV** (out-of-vocabulary) 문제를 **원천적으로 회피** — n-gram은 “보지 못한 단어 = 확률 0”이었으나, BPE 모델은 보지 못한 단어라도 아는 조각으로 분해해 예측 가능.
- 17.1절 전체의 흐름 = **어휘의 유한성**이라는 한계를 단어 → 임베딩 → 조각의 순서로 우회해 온 역사.

BPE 손계산: 세 줄 corpus에서 6회 병합

corpus: “서울의 겨울은 춥다 / 서울의 봄은 따뜻하다 / 파리의 겨울은 춥다” (여백 없이 문자 단위 → 25개 글자, 서로 다른 13종)

병합	병합 내용(빈도)	토큰 수	어휘 사전 크기
처음	각 문자	25	13
1	서+울 → 서울 (2회)	23	14
2	서울+의 → 서울의 (2회)	21	15
3	겨+울 → 겨울 (2회)	19	16
4	겨울+은 → 겨울은 (2회)	17	17
5	겨울은+춥 → 겨울은춥 (2회)	15	18
6	겨울은춥+다 → 겨울은춥다 (2회)	13	19

- **반복되는 조각이 먼저 묶인다** — “서울의” · “겨울은”는 각 2회 등장, 한 번만 나온 조합보다 먼저 한 토큰. 빈도 통계만으로 자연스러운 단위 발견.
- **경계는 모른다** — “겨울은 춥다”라는 문장 전체가 ‘겨울은춥다’ 하나로 묶임.

corpus가 수조 토큰이면 형태소 단위(의, 은, ㄴ, ㅏ...)로 수렴. 이토록 작은 corpus에서는 과하게 큰 토큰까지 만들게 된다 — **BPE는 규모에 의존한다.**

손으로 한 번: n-gram으로 다음 토큰 분포 세기

- 신경망에 들어가기 전에, 가장 단순한 **n-gram 언어모델**으로 뼈대를 손으로 만든다. (세는 데 집중하기 위해 여기서는 **단어 단위** 토큰 — `.split()`으로 여백에서 나눈 것.)
- “다음 토큰의 확률”을 **앞의 $n - 1$ 개 토큰만으로 조건부로** 근사:

$$P(x_t \mid x_1, \dots, x_{t-1}) \approx P(x_t \mid x_{t-n+1}, \dots, x_{t-1})$$

- $n = 1$: 앞 토큰을 보지 않는 무조건분포(어휘 전체에 나눈 단어 빈도). $n = 2$: **bigram**.
- 학습은 단순히 **세는 것**: “서울의”가 나온 모든 위치에서 바로 다음 토큰을 센다.

bigram 모델: “서울의” 뒤에?

```
from collections import defaultdict
import math

def next_token_distribution(corpus_tokens, context_word):
    # context_word 등장한 모든 위치에서, 바로 다음 토큰을
    # 다음 단어{: 확률} 딕셔너리로 반환 (bigram 모델, n=2)
    counts = defaultdict(int)
    for i in range(len(corpus_tokens) - 1):
        if corpus_tokens[i] == context_word:
            counts[corpus_tokens[i + 1]] += 1
    total = sum(counts.values())
    if total == 0:
        return {} # 문맥이 한번도 등장한 적 없으면 예측 근거 없음
    return {w: c / total for w, c in counts.items()}

corpus = "서울의 겨울은 춥다 서울의 봄은 따뜻하다 파리의 겨울은 춥다".split()
print(next_token_distribution(corpus, "서울의"))
# 겨울은 {'': 0.5, 봄은 '': 0.5} -- 서울의 "" 뒤에는 겨울 봄이 / 각 50%
```

실제 LLM의 다음 토큰 분포도 이 딕셔너리의 “어휘 사전 전체 버전”에 불과 — 3만~20만 토큰을 키로 하는 확률 테이블. 다만 값들이 **문장 전체**에 조건부이고 **학습으로** 결정된다는 두 차이뿐.

n-gram의 두 한계, 그리고 PPL을 손으로

- **첫째, 과소표본(sparsity):** “파리의 겨울은” bigram이 corpus에 한 번만 있으면 $P(\text{춡다} \mid \text{파리의, 겨울은}) = 1.0$ — **관찰 1개**에 기반한 확신. 반대 증거 한 줄(“파리의 겨울은 덥다”)만 추가해도 1/2로 반토막.
- **둘째, OOV:** corpus에 한 번도 없는 토큰(“뉴욕의”)에는 **빈 딕셔너리**를 반환 — 근거 자체가 없음.
- bigram($n = 2$)의 **퍼플렉시티**: 각 위치 “실제로 온 다음 토큰”의 확률을 곱한 뒤 N 제곱근:

$$\text{PPL} = \exp \left(-\frac{1}{N} \sum_{t=2}^N \log P(x_t \mid x_{t-1}) \right) = \left(\prod_{t=2}^N P(x_t \mid x_{t-1}) \right)^{1/N}$$

- 이 corpus(9개 토큰)에서 unigram($n = 1$)과 대조: **문맥을 볼수록 PPL이 낮아짐** — 실습에서 unigram ≈ 5.67 vs bigram ≈ 1.19 .

이 두 한계(데이터 부족한 데 확신, 못 본 토큰엔 침묵)가 바로 **더 긴 문맥을 보는 신경망 + 조각 기반 BPE**의 존재 이유.

좋은 언어모델 = PPL이 낮은 모델.

softmax: 원점수를 확률로 — 숫자로

- 모델이 산출하는 것은 확률이 아니라 **원점수** (logit) $z_j \in \mathbb{R}$. **softmax**(Ch. 2.3 시그모이드의 다클래스 일반화):

$$P(x_t = j) = \frac{e^{z_j}}{\sum_k e^{z_k}}$$

- 왜 e^z 로 정규화: (1) 모든 확률 0~1, 합 1, (2) **원점수 차이가 그대로 확률 비율로**:
 $z_a - z_b = 1$ 이면 $P(a)/P(b) = e \approx 2.7$ 배.
- **숫자로**: $z = (4, 1, 0, -3)$ 이면 $e^z \approx (54.6, 2.72, 1.0, 0.05)$, 합 $\approx 58.37 \rightarrow$ 확률 $\approx$ **(0.935, 0.047, 0.017, 0.001)** — 첫 토큰에 93.5%가 모인 “확신 있는 예측”.
- 반면 $z = (1, 1, 1, 1)$ 이면 (0.25, 0.25, 0.25, 0.25) — **완전한 무지(무차별 우연)**.
- **shift-invariance**: 모든 logit에 같은 상수를 더해도 분포는 변하지 않음 — 확률은 차이로 결정. 이 벡터를 실제 정답과 비교하는 것이 교차 엔트로피 손실.

온도(Temperature): 확신도를 조절하는 손

- softmax에 온도 $T > 0$ 을 넣으면:

$$P(x_t = j) = \frac{e^{z_j/T}}{\sum_k e^{z_k/T}}$$

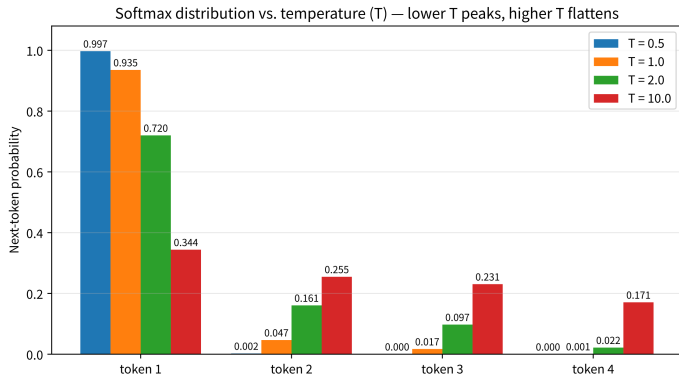
(원점수를 T 로 나눠준 뒤 softmax)

- $T < 1$: 차이 **늘어** → 분포 **점근 (peaky)**, 확신 강해짐. $T > 1$: 차이 **줄어** → **평탄(flatter)**, 더 무작위. $T \rightarrow \infty$: 등확률.
- $z = (4, 1, 0, -3)$ 예로: $T = 0.5 \approx (0.997, 0.003, 0.000, 0.000)$ 거의 최고 토큰만 / $T = 2.0 \approx (0.720, 0.161, 0.098, 0.022)$ 확신 완화 / $T = 10.0 \approx (0.344, 0.255, 0.231, 0.171)$ 거의 무작위.

실전 LLM 채팅봇의 temperature 파라미터

“창의적인 글” → $T > 1$ (확률을 넓혀 덜 예상적인 토큰도) / “사실·코드” → $T \rightarrow 0$ (가장 확신 있는 토큰만). 동일한 softmax 분포를, “토큰을 고를 방법”에 따라 확신도를 조절하는 것 — 17.2절 샘플링 전략과 17.3절 포스트트레이닝(고온 샘플링으로 다양한 후보를 만들어 비교)의 기초.

온도의 그림



- “창의적”과 “정확”을 한 파라미터로 **연속적으로** 조절.
- 17.2절에서 이 분포 위에 **top-k, top-p** 샘플링 전략이 올라온다.

$T = 0.5, 1.0, 2.0, 10.0$ 에 따른 softmax 분포: T 가 작을수록 확신이 한 토큰에 몰리고(점근), 커질수록 네 토큰에 고르게 퍼져(평탄) 무작위성이 커진다.

교차 엔트로피 손실: “정답이 받은 확률”을 최대화

- 모델이 토큰 j 에 확률 p_j 를 부여했고, **실제로 정답이 j 였을 때** 그 한 토큰의 손실:

$$\mathcal{L}_{\text{token}} = -\log p_j$$

- 세 가지 값으로 읽기: $p_j = 0.9 \rightarrow -\log 0.9 \approx 0.105$ (거의 맞힘, 거의 벌 없음) / $p_j = 0.5 \rightarrow 0.693$ (동전 던지기 수준, 중간 벌) / $p_j = 0.001 \rightarrow 6.908$ (거의 0 확률에 정답, 큰 벌).
- **왜 로그인가**: (1) 문장 확률 = 토큰 확률의 곱은 길어질수록 underflow — 로그가 곱을 합으로. (2) **감소효용**: 확률 0.1→0.35(3.5배)는 손실 1.25를 줄이지만 0.5→0.75(1.5배)는 0.41밖에 줄이지 못함.
- 정답 확률이 0.99까지 모이면 그래디언트가 거의 0이 되어 크게 갱신하지 않고 수렴 — **softmax + 교차 엔트로피가 안정적으로 학습되는 이유**(Ch. 2.3 로지스틱회귀와 같은 논리).

문장 손실과 PPL: 하나의 사슬

- 문장(minibatch) 전체 손실은 토큰 손실의 **평균**:

$$\mathcal{L} = -\frac{1}{N} \sum_{t=1}^N \log P(x_t \mid x_1, \dots, x_{t-1})$$

- 퍼플렉시티는 이 손실의 지수:

$$\text{PPL} = e^{\mathcal{L}}$$

- $\mathcal{L} = 2.3$ 이면 $\text{PPL} = e^{2.3} \approx 10$ — “평균 10개의 균등한 후보 중 하나를 고르는 것과 같음”.
- 교차 엔트로피 $\leftrightarrow$ PPL, 로그 $\leftrightarrow$ 지수 — 앞 softmax의 “원점수 차이가 확률 비율로” 변환과 같은 수학적 구조.

logits $\rightarrow$ **확률** $\rightarrow$ **손실** $\rightarrow$ **PPL**이 하나의 사슬로 이어져 있다 — 17.3절 보상모델 손실 (또다시 $-\log$ 형태)까지 자연스럽게 이어진다.

역사: n-gram에서 신경망까지

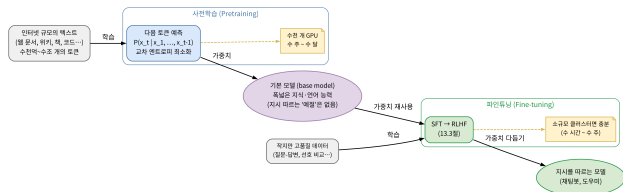
- “다음 단어를 맞히기”는 현대 LLM이 발명한 것이 아님. **1940년대 새년**(Claude Shannon)의 정보이론 논문에서 영어의 “다음 단어 예측 엔트로피”를 추정할 때 쓰던 것이 바로 n-gram — 사람에게 문장을 읽고 다음 단어를 맞히게 해 기계의 상한을 재는 실험까지.
- **1990~2000년대**: n-gram($n = 3 \sim 5$)이 음성인식·기계번역의 표준. 그러나 **두 벽**: (1) **문맥 길이** — 앞의 $n - 1$ 개만 봄, (2) **OOV와 조합 폭발** — V^n ($V = 50,000, n = 3$ 이면 1.25×10^{14} 개) 중 대부분 0 확률 → **스무딩**(등장하지 않은 조합에도 작은 확률 분배) 필수.
- **2013년 word2vec**(Mikolov et al.): “단어의 의미 = 주변 단어 분포”라는 분포 가설을 **임베딩**으로 효율 학습(Ch. 14).
- **2017년 Transformer**(Ch. 12): 자기-어텐션으로 “문장 전체의 모든 토큰을 동시에 조건부”로 보고, **스케일만** 키워도 성능이 계속 올라감 → 80년 된 과제가 **LLM**이라는 새로운 현상(Brown et al., 2020)을 만들어냄.

사전학습 vs 파인튜닝

	사전학습(Pretraining)	파인튜닝(Fine-tuning)
데이터	인터넷 규모의 방대한 텍스트	특정 목적에 맞는 비교적 적은 데이터
목표	다음 토큰 예측(비지도)	특정 작업(대화, 지시 따르기 등)에 맞춘 지도학습
결과	폭넓지만 다듬어지지 않은 능력	목적에 맞게 다듬어진 행동

- 사전학습: 계산 비용 **막대** — 수천 GPU, 몇 주~몇 달. 파인튜닝: 상대적으로 적은 자원.
- 핵심: “**기초를 다시 배우지 않고, 이미 아는 것을 특정 방향으로 다듬는다**” — Ch. 10.3의 **전이학습**(사전학습 CNN 앞쪽 층 고정, 마지막 층만 재학습)과 **정확히 같은 논리**. 이미지 대신 텍스트, 합성곱 필터 대신 Transformer 가중치.

2단계 파이프라인과 비용의 비대칭성



- 사전학습 비용은 **데이터의 양**(수조 토큰)에 비례 → **“한 번만”**.
- 파인튜닝 비용은 **작업의 수**(대화, 코드, 요약, 역할극...)에 비례 → **“작업마다”**.

인터넷 규모 텍스트로 다음 토큰 예측(수천 GPU, 수 주~수 달) →
 기본 모델 → 질문-답변/ 선호 데이터로 SFT·RLHF
 파인튜닝(소규모 클러스터, 수 시간~수 주) → 지시를 따르는
 모델.

하나의 기본 모델 + 여러 파인튜닝

“여러 개의 별도 사전학습 모델”보다
 효율적인 이유 = 이 **비용의
 비대칭성**(데이터 비례 vs 작업 비례).

왜 앞쪽은 고정한 채 뒤쪽만 다듬나

- CNN에서: **앞쪽 층**(저수준: 모서리, 색)은 이미지에 → 고정, **마지막 층**(고수준: 고양이 인가 개인가)만 재학습.
- LLM에도 같은 층별 역할 분화:
 - ▶ **앞쪽 Transformer 층**: 토큰 주변의 문법/의미 패턴 — **문언어적(low-level linguistic) 특징**
 - ▶ **뒤쪽 층**: 어떤 내용/톤/형식의 답변이 적절할 것인가 — **고수준 의미/행동 특징**
- 사전학습이 앞쪽에 폭넓은 지식을 이미 집어넣었으므로, 파인튜닝은 **뒤쪽 층 중심**으로 “어떤 상황에서 어떤 답변을 할 것인가”라는 **행동 매핑만** 다듬으면 된다 → 전체 파라미터의 소수%(LoRA, 17.3절) 만 갱신해도 효과가 큰 이유.

앞쪽을 재학습하면(= 사전학습을 다시 하면) 축적한 지식을 파괴하면서 작업에 맞지 않는 방향으로 편향시킬 위험 — 파인튜닝에서는 앞쪽을 **고정하거나 아주 작은 학습률로만** 갱신하는 것이 표준.

자주 묻는 질문 (17.1)

- **Q. PPL이 낮으면 항상 더 똑똑한가?** PPL은 “다음 토큰을 얼마나 **확신 있게** 예측하는가”만 잰다 — 사실 정확성·추론은 **별도로** 평가. 같은 문장만 반복하는 데이터로 학습한 모델은 PPL이 매우 낮을 수 있지만 쓸모없다. **PPL과 작업 성능 벤치마크를 항상 함께 본다.**
- **Q. 데이터 양과 모델 크기의 관계?** **스케일링 법칙**(Kaplan et al., 2020): 파라미터 수 · 데이터 양 · 계산량 중 하나만 키우면 금방 한계 — 셋을 **적절한 비율로 함께** 키워야 계속 좋아진다는 경험적 관계. “얼마나 크게, 얼마나 많이 모아야 하는가”의 실무 지침.
- **Q. “다음 토큰 예측”은 너무 단순한 것 아닌가?** 목표는 단순하지만 **분포가** 단순하지 않다. “1 + 1 = ?” 뒤에 “2”가 확실히 오는 분포를 만들려면 **딥넷의 규칙을 일반화**해야 한다 — **단순한 목표 + 복잡한 분포 = 일반화되는 능력**이 바로 **신흥 능력**(emergent ability, Wei et al., 2022)의 기원. 17.2절 CoT(Wei et al., 2022)가 이 능력을 **의식적으로 유도**.
- **Q. BPE 어휘 크기는 어떻게 정하나?** 3만~20만 표준. 작으면 토큰이 길어져 **문맥 길이 제한**(Ch. 12 $O(n^2)$ 어텐션), 커지면 **임베딩 파라미터**($V \times d$; $V = 200,000, d = 4096$ 이면 약 8×10^8) 커짐 — **토큰 길이 vs 임베딩 비용** 트레이드오프. 언어/도메인에 따라 병합 순서가 달라져 **같은 문장도 모델마다 다른 토큰 수**로 측정.

자주 하는 실수 (17.1)

- “다음 토큰 예측”을 “다음 단어 예측”으로만 읽기 —
 - ① BPE의 조각 단위(“서울의” 예처럼 한 토큰이 단어의 일부일 수 있음)를 놓치고,
 - ② 문맥이 앞의 모든 토큰 전체(직전 단어가 아님)라는 것을 잊는다.
- (a) “단어를 모르니 OOV 문제가 있다”는 오해 — **BPE가 이미 원천 해결**. (b) “직전 단어만 보고 예측한다”는 오해 — 자기-어텐션이 **문장 전체**를 조건부로 봄(Ch. 12).
- **PPL과 작업 성능을 혼동** — PPL은 “확신 있게 예측하는가”를 잴 **사전학습 단계의 내부 지표**. 17.2절 **환각**(hallucination)은 **PPL이 낮아도** 발생할 수 있다.

두 구분이 “왜” BPE와 Transformer를 쓰는지의 핵심

토큰 $\neq$ 단어, 문맥 = 앞의 모든 토큰. PPL = 언어의 그럴듯함, 벤치마크 = 작업의 정확성.

연습문제: softmax · CE · PPL을 완성하라

```
import math

def softmax(z):
    # z: 원점수리스트 -> 확률분포합 ( 1)
    # 수치( 안정: max(z)를 먼저빼고지수를취하라 )

def cross_entropy_loss(probs, true_idx):
    # -> -log(probs[true_idx]) 반환

def perplexity(token_losses):
    # -> exp(평균( 손실)) 반환

# 검증본문 ( 예: z=(4,1,0,-3))
p = softmax([4.0, 1.0, 0.0, -3.0])
assert abs(p[0] - 0.935) < 1e-3          # [0.935, 0.047, 0.017, 0.001]
assert abs(cross_entropy_loss(p, 0) - 0.067) < 1e-3
print(f"{perplexity([0.067, 0.693, 3.912]):.2f}") # e^1.557 ~= 4.75
```

“확률이 높을수록 손실이 작아진다”가 숫자로 확인되어야 한다. 손계산: (a) bigram $P(\text{출다} \mid \text{파리의, 겨울은})$ vs $P(\text{출다} \mid \text{서울의, 겨울은})$, (b) unigram $P(\text{출다})$ (힌트: “출다” 2회, 토큰 9개). shift-invariance: 모든 logit에 +100를 더해도 분포 불변.

17.2 프롬프팅

프롬프팅: 다시 학습시키지 않고 행동을 바꾸기

- 파인튜닝은 모델의 **가중치**를 바꾸는 작업.
- **프롬프팅**(prompting)은 가중치를 **전혀** 바꾸지 않고 **입력(지시문, 예시)**만으로 출력을 원하는 방향으로 유도.
- 세 가지 기법:
 - ▶ **Zero-shot**: 예시 없이 지시문만 — “이 문장을 한국어로 번역해줘: ...”
 - ▶ **Few-shot**: 입출력 예시 몇 개 포함시켜 패턴 파악 후 따라하게 (Brown et al., 2020).
 - ▶ **Chain-of-Thought(CoT)**: 최종 답 전에 **중간 추론 과정**을 출력하도록 유도.

스펙트럼: “프롬프트에 얼마나 보여줄 것인가”

zero-shot = “**무엇을** 할지”만 말하고, few-shot = “어떤 **형식**으로” 하는지 보여주고, CoT = 어떤 **과정**으로 하는지까지 보여준다. 공통: **가중치를 건드리지 않는다** — 학습 업데이트(그래디언트, 역전파)가 한 번도 일어나지 않고 **추론(inference)** 시점에 프롬프트만으로 동작이 바뀐. 파인튜닝 = “모델을 **바꾸는**” / 프롬프팅 = 그 자리에서, **즉각적으로**, 원상복구 가능하게 “모델의 사용법을 바꾸는”.

왜 “예시 몇 개”가 새 작업을 가르치는가

- 예시만 포함시키면 새 형식의 작업을 수행하게 된다는 사실(few-shot)은, **사전학습이 얼마나 폭넓은 능력을 압축해뒀는가**의 증거.
- 사전학습 데이터에는 “질문-답변”, “예시-패턴” 형태가 이미 **무수히** 포함 — 프롬프트에 비슷한 패턴을 보여주면 그 패턴을 이어가는 방향으로 다음 토큰을 예측.
- 즉: **새 지식을 학습하는 것이 아니라**, 이미 갖고 있는 능력 중 **어떤 것을 꺼내 쓸지**를 프롬프트가 가리키는 것에 가까움.
- 17.1 관점: LLM 출력은 $P(x_t \mid x_1, \dots, x_{t-1})$ — **지금까지 프롬프트에 포함된 모든 토큰에 대한 조건부 확률**. 예시 3개를 붙이면 조건이 **그 예시를 포함한 전체 시퀀스**.
- Ch. 12.2 어텐션 관점: 질문 토큰이 다음 단어를 고를 때 예시 부분을 **실제로 “살펴보는”** 것 — 그래서 예시를 바꾸면 예측이 바뀜.
- “few-shot = 모델이 예시로 **학습한다**”는 표현은 **정확하지 않다** — 컨텍스트 안의 토큰이 **조건으로** 작용할 뿐, 파라미터에 새겨지는 것이 아님. 프롬프트를 바꾸면 **즉시, 완벽하게** 원래대로 복귀.

조건으로 삼는 토큰이 늘면 어텐션 계산량이 n^2 으로 커진다는 비용 함의도 따라옴(Ch. 12.3).

장난감 모델로 확인: 예시가 있으면 더 잘 맞는가

```
# zero-shot: 사과는이라는 "" 패턴을프롬프트에서한번도보여주지않음
zero_shot_context = "오늘 기분이좋다사과는 ".split()
print(next_token_distribution(zero_shot_context, "사과는"))
# {} -- 아무근거가없어예측불가

# few-shot: "는00 이다00" 패턴예시개 2 + 질문
few_shot_context = "사과는 과일이다바나나는과일이다자동차는기계이다사과는 ".split()
print(next_token_distribution(few_shot_context, "사과는"))
# 과일이다{' ': 1.0} -- 프롬프트안의예시가곧바로예측근거가됨

# 한계: 비행기는은"" 예시에서나온적없으니 , 패턴을알더라도 {}
```

zero-shot: “사과는”가 문맥에 없어 예측 불가(빈 딕셔너리). few-shot: 예시가 이미 포함되어 있어 그 패턴을 근거로 100% 확신. 이 장난감 모델은 진짜 LLM의 **훨씬 단순화된 버전** — 진짜 LLM은 의미적으로 유사한 새 단어에도 일반화. 하지만 “**프롬프트에 넣은 토큰이 곧바로 다음 예측의 조건이 된다**”는 기계적 원리 하나 — 그 원리가 **few-shot의 전부**. 노트북에서는 temperature까지 붙여 “같은 프롬프트에서 왜 매번 다른 답이 나는가” 확인.

Few-shot 예시의 실무 요령

- **형식 일관성**: 예시들이 **같은 형식**(같은 라벨, 문장 구조, 언어)을 유지해야 한다. “네/아니오” 5개에 “yes/no”가 섞이면 출력이 흔들릴 수 있다.
- **라벨 순서 효과**: 예시의 라벨 배열 자체가 출력을 바이어스 — “majority가 ’그렇다’였다”는 패턴이 출력도 “그렇다” 쪽으로 기울게 하는 현상이 실제로 관측(Min et al., 2022). 정답 분포가 편향된 작업(위반이 드문 판정)에는 라벨 순서를 의도적으로 섞어야 할 경우.
- **1~4개가 일반**: 대부분의 작업에서 1~4개 예시로 충분.
- 양과 지속성의 차이: 17.3절 SFT/RLHF가 수천~수만 개 예시를 **가중치**에 새긴다면, few-shot은 지금 질문 한 번에 대해서만 **컨텍스트** 안에 넣는 방법.

Chain-of-Thought: “단계별로 생각해봐”

- 2022년 **Wei et al.**(Google Brain)에서 체계적으로 보고: “ 27×43 을 계산해라”에 “단계별로 풀어봐”(think step by step) 한마디를 붙이면 정답률이 현저히 올라감.
- 같은 해 **Kojima et al.**는 더 극단적: “Let’s think step by step.”라는 문장 하나를 질문 뒤에 붙이기만 해도(예시 없이, 지시 한 줄) 추론 성능이 크게 좋아짐.
- 왜 작동하는가 — 17.1 조건부 확률 관점에서 보면 직관적:

- ▶ 최종 답을 **한 번에** 예측하게 하면

$$P(\text{최종답} \mid \text{질문})$$

— 중간 단계가 하나도 없는 **어려운** 조건부 확률을 출력해야 함.

- ▶ “단계별로”라고 요청하면, 그 하나의 어려운 확률이 **여러 개의 쉬운 조건부 확률의 곱**으로 분해:

$$P(m_1 \mid q) P(m_2 \mid q, m_1) P(\text{최종답} \mid q, m_1, m_2, \dots)$$

각 m_i 는 “지금까지 쓴 내 추론”이라는 풍부한 조건 아래 예측 → 한 단계당 난이도 대폭 하락.

CoT의 메커니즘: 숫자로

직접 한 방에 답

- 중간 결과를 조건으로 쓸 수 없어, 각 부분 추론을 **한 번에** 맞히기(하나당 0.5):

$$0.5^3 = 0.125$$

CoT로 단계별로

- 각 중간 결과가 다음 단계의 **조건**이 되어 하나당 0.9:

$$0.9^3 = 0.729$$

핵심 메커니즘이 숫자로

모델이 스스로 쓴 중간 결과가, 다음 토큰 예측의 조건이 된다 — 쓰느냐 마느냐가 부분 추론 하나당 0.5에서 0.9로 끌어올린다.

어텐션으로: 최종 답 토큰이 앞의 중간 추론 토큰을 실제로 “본다”. 단계가 늘수록 격차 기하급수: 2단계 0.81 vs 0.25 / 5단계 0.59 vs 0.03 / 10단계 0.35 vs 0.001. 노트북 4,000회 시뮬레이션으로 ≈ 0.125 vs ≈ 0.73 직접 확인.

손으로: 47×83

- 직접 프롬프트(“ 47×83 은 얼마인가? 답만 말해줘”): 모델이 “3901”이라는 4자리를 아무 중간 작업 없이 내뱉어야 한다.
- CoT 프롬프트(“단계별로 계산해줘”):

$$47 \times 83 = 47 \times (80 + 3) = 47 \times 80 + 47 \times 3 = 3760 + 141 = \mathbf{3901}$$

- 각 하위 단계(47×80 , 47×3 , 덧셈)는 훨씬 쉬운 다음 토큰 예측. 중간 단계가 **프롬프트 안의 컨텍스트로 남아** 있어서, 마지막 덧셈은 자기 자신의 정확한 중간 결과(“ $3760 + 141$ ”)를 조건으로 한다.

중간 단계 하나가 틀리면 오차가 다음 단계로 **전파** — “단계별로”는 정확도를 높이면서도 **오류도 단계마다 쌓이는** 구조. 이 “오류 전파”를 줄이려는 연구 → 17.3절 포스트트레이닝이 그 다음 단계.

생성 파라미터: temperature, top-k, top-p

- 17.1: 각 단계 출력이 어휘 전체에 대한 **확률분포**(logits에 softmax). 그 분포에서 실제로 어떤 토큰을 **고를 것인가**를 정하는 것이 **생성 파라미터**.
- Greedy(탐욕법)**: 매 단계 **확률 최고** 토큰($\arg \max$). 결정적이고 빠르지만, “현재 최고”만 보면 장기적으로 **반복**에 빠지기 쉬움.
- Temperature T** : logits를 T 로 나눈 뒤 분포:

$$P(x_t = i \mid x_{<t}) = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

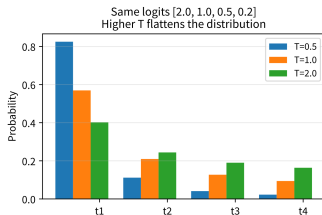
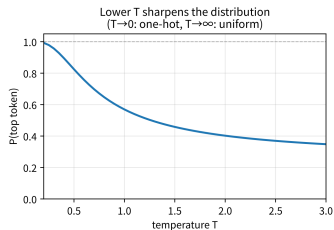
$T < 1$: **가파라짐**(최상위 독점) / $T > 1$: **평탄해짐**(후순위에도 실질적 확률). $T \rightarrow 0$ 은 greedy, $T \rightarrow \infty$ 는 균등분포.

- top-k**(Fan et al., 2018): 확률 **상위 k 개** 토큰에서만 샘플링(나머지 확률 0). $k = 1$ 이면 greedy와 같음.
- top-p, nucleus sampling**(Holtzman et al., 2019): 누적 확률이 p 에 도달하는 **최소** 토큰 집합 안에서 샘플링. 분포가 **가파르면** 집합이 **작아지고**, **평평하면 커짐** — **상황에 맞춰 “선택지 폭”을 자동 조절** (top-k와의 차이이자 장점).

온도 체감: logits [2.0, 1.0, 0.5, 0.2]

T	분포	최상위 확률
0.5	(0.825, 0.112, 0.041, 0.023)	0.825 — 매우 집중
1.0	(0.569, 0.209, 0.127, 0.094)	0.569
2.0	(0.402, 0.244, 0.190, 0.164)	0.402 — 훨씬 평평

temperature — a parameter controlling the 'width of choices'



같은 logits에 $T \in [0.2, 3.0]$ 적용 시 최상위 확률 곡선(좌:
 $T \rightarrow 0$ 일수록 1로 수렴)과 $T = 0.5/1.0/2.0$ 분포 막대(우).

실무 감각

- **결정적** 작업(사실 확인 · 추출 · 코드):
 T 낮게(0~0.3)
- **창의적**
작업(시 · 카피 · 브레인스토밍): T
높게(0.7~1.0 이상)

프롬프트가 “행동을 바꾼다”는 것의 실제 의미

- 프롬프팅이 “능력을 켜고 끄는 스위치”처럼 느껴지는 것은, 사실 17.3절의 **SFT**가 **미리 깔려** 있어서다.
- 사전학습만 거친 원시 모델 = “지속해서 텍스트를 **이어가는**” 모델 → SFT에서 “사람의 지시를 **따르는 형식**”으로 다듬어짐.
- “이 JSON으로 답해줘”, “200자 이내로”가 **즉답**으로 작동 = **사전학습(폭넓은 능력) + SFT(지시 따르는 형식) + 프롬프팅(무엇을 쓸지 가리키는 입력)**의 세 겹. 프롬프팅은 그 세 겹 중 **입력 레벨**의 기법.
- **RAG**(Retrieval-Augmented Generation, Lewis et al., 2020): 질문과 관련된 문서를 **검색해서** 프롬프트에 붙여 “이 문서에 기반해 답하라”.
- **에이전트**(agent, ReAct: Yao et al., 2022): 모델이 검색·계산기·코드 실행 같은 **도구를 스스로 호출**하도록 프롬프트 설계.
- 둘 다 “파라미터 안에 모든 지식을 우겨넣는” 대신 **필요한 정보를 컨텍스트에 공급** — 이 메커니즘이 바로 **프롬프팅 그 자체**(17.3절 에서 상세히).

프롬프트의 구조: system / user / few-shot

- **System message**(역할): “너는 ...전문가다. 항상 JSON으로 답한다.” — 대화 전체에 걸친 기본 전제.
- **User message**(질문): 이번 요청 자체.
- **Few-shot 예시**(선택): 질문 앞에 붙는 입출력 쌍.

마법이 아니라 다음 토큰 예측

이 구조 자체는 Ch. 12의 “입력 시퀀스”일 뿐 — 모델은 system/user를 **구분해서** 처리하는 것이 아니라, **모든 토큰을 같은 다음 토큰 예측 조건으로** 취급. “역할”이 작동하는 것도 그 텍스트가 **조건에 포함**되기 때문. (그래서 system message를 “지키지 않으면”이라는 표현이 애매 — 모델에게는 그저 긴 컨텍스트의 앞부분.)

환각(Hallucination): 왜 생기는가

- LLM은 **사실이 아닌 그럴듯한 문장**을 자신 있게 만들어내기도 한다(**환각**, Ji et al., 2023).
- 원인: “다음 토큰으로 **그럴듯한 것**”을 예측하도록 학습됐을 뿐, **사실 여부를 검증하는 과정이 내장돼 있지 않기 때문**.
- 17.1과 연결: 손실을 최소화하는 모델은 “실제 데이터에서 **흔히** 이어지는 토큰”을 잘 예측하는 모델. “**흔히 이어진다**”와 “**사실이다**”는 다른 개념 — 인터넷 텍스트에는 잘못된 정보·과장·허구 도 섞여 있음.
- “그렇듯하다”는 것은 모델의 **자신감**(토큰 확률)과도 별개 — 틀린 사실을 **매우 높은 확률로** 출력할 수 있다.

흔한 유형: (1) 없는 사실을 지어냄 (가짜 인용, 가짜 법조항), (2) 있는 사실을 **틀리게** 기억(인물·날짜·숫자 오류), (3) **자기 모순**(같은 대화 안에서 앞뒤 다른 진술) — (3)은 빈도 세기 장난감 모델에서는 원리적으로 생기지 않으나, 실제 LLM은 sampling 개입 + 긴 컨텍스트로 앞에서는 A, 뒤에서는 B라고 말할 수 있다.

환각을 줄이는 실무 수단

- **낮은 temperature** — “그렇듯하지만 드문” 분기에서 벗어나게. (단, $T = 0$ 이 환각을 없애는 것은 아님 — 결정적일 뿐, 정확한 것은 아님.)
- **RAG** — 답의 근거가 될 문서를 프롬프트에 직접 넣고 “이 문서에 기반해 답하라”. 모델이 **가진** 문맥 안에서 답을 만들어야 하므로 지어낼 여지 감소.
- **“모르면 모른다고 해” 지시** — 효과는 있으나 **보장은 아님**.
- **Self-consistency**(Wang et al., 2022) — CoT를 **여러 번** 샘플링($T > 0$)해서 나온 최종 답들 중 **다수결** 채택. 한 번의 추론이 틀려도 다수가 맞으면 오차 상쇄.

“중간 결과를 조건으로 쓴다”(CoT)와 “여러 경로를 탐색한다”(sampling)를 **조합**한 기법. 환각을 **구조적으로** 줄이려면 근거 공급(RAG) · 다수결(self-consistency) · **외부 검증**을 프롬프트 바깥에서 덧붙이는 것이 훨씬 효과적.

자주 하는 실수 (17.2)

- “프롬프트 = 질문 한 줄”로 쓰기 — 역할 지정 없으면 모델이 “어떤 식의 번역인가”를 추측. 프롬프트는 프로그램: 역할·작업·형식·예시를 명시할수록 출력이 예측 가능.
- few-shot 예시의 형식을 대충 — 3개 중 형식이 섞이면 어떤 형식을 따라야 할지 혼란. 형식은 모두 동일해야.
- “프롬프트가 길수록 좋다” — 컨텍스트에 한계(n^2 계산량)가 있고, 지나치게 긴 프롬프트에서는 중간 부분 정보의 영향이 오히려 줄어든다 (**lost in the middle**, Liu et al., 2023). 핵심 지시문은 앞이나 뒤에 두는 것이 안전.
- “temperature 0이면 환각이 사라진다” — $T = 0$ 은 결정적을 의미할 뿐, 정확은 아님. 틀린 사실을 확신 있게, 매번 같은 방식으로 반복하는 것이 $T = 0$ 의 환각.
- “단계별로 생각해봐”를 모든 작업에 붙이기 — “오늘 날씨는?” 같은 단순 사실 조회에 CoT를 붙이면 답은 그대로인데 토큰 수(= 비용 과 지연)만 늘어남. 단계가 있는 문제(산술, 논리, 계획)에서만 효과.

확인 문제 (17.2)

- **1 (temperature 손 계산)** 어휘 3개 $\{A, B, C\}$ 의 logits가 $[3.0, 1.0, 0.0]$. $T = 1$ softmax 분포를 구하고(힌트: $e^3 \approx 20.1, e^1 \approx 2.72, e^0 = 1$), $T \rightarrow 0$ 일 때 분포가 어떤 형태로 수렴하는지 한 문장으로.
- **2 (개념)** 왜 $T > 0$ 샘플링이면 완전히 같은 프롬프트라도 매번 다른 답이 나올 수 있는가 — “다음 토큰 예측이 **확률분포**를 반환한다”는 17.1의 사실로부터 2~3문장으로 유도. “매번 다른 답”이 **불편하지 않은 / 불편한** 작업 각 하나.
- **3 (코딩)** 장난감 모델로 “OO는 OO이다” 패턴 예시 3개 + “사과는” 질문: zero-shot / two-shot / few-shot 출력을 비교해 “예시 개수가 예측 확률에 어떻게 영향을 미치는가”를 숫자로 확인.

자주 묻는 질문 (17.2)

- **Q. Few-shot 예시는 몇 개?** 정해진 답은 없다. 1~2개로 충분한 경우도 있고, 복잡할수록 더 많은 예시가 도움. 다만 어텐션 n^2 계산량을 고려해 무한정 늘리는 것은 비용·속도 트레이드오프 — 실무는 1~4개에서 시작해 “더 늘렸을 때 실제로 오르는지” 확인.
- **Q. CoT가 왜 정확도를 높이나요?** 최종 답을 곧바로 예측하게 하면 중간 추론 없이 “그럴듯한 답”을 내놓아야 하지만, 중간 과정을 먼저 출력하면 그 과정 자체가 **다음 토큰의 조건**이 되어 “참고 자료”로 활용 가능. 0.9^3 vs 0.5^3 숫자 예가 바로 이 메커니즘의 단순 수치 모델.
- **Q. 프롬프팅도 “학습”의 일종인가?** 아니요 — 학습 = 가중치 갱신(영구), 프롬프팅 = 가중치 고정 + 컨텍스트 조건 변경(일시, 그 요청 한 번). “in-context”의 in = **컨텍스트 윈도우 안에서**. 프롬프트를 빼면 완벽하게 원래대로.
- **Q. zero-shot이 항상 열등인가?** 아니요 — **흔한** 작업(번역, 요약, 일반 분류)에서는 사전학습에서 수십억 번 봤기에 충분. few-shot이 특히 유리한 경우: (a) **비표준 형식**, (b) **다의적 라벨**, (c) **새로운 도메인**. 예시를 너무 많이 넣으면 (i) n^2 비용, (ii) **예시 편향** 전이. **“더 많은 예시 = 더 좋아”는 항상 성립하지 않는다.**
- **Q. “사실이 아니야, 틀리지 마”라고 쓰면 환각이 줄까?** 약간은 줄 수 있다(더 신중하게 라는 형식에 SFT됐을 수 있음). 하지만 **보장은 아님** — 모델은 사실 검증기가 아니라 **그럴듯한 토큰 생성기**. 환각을 **구조적으로** 줄이려면 근거 공급(RAG)·다수결(self-consistency)·**외부 검증**을 프롬프트 바깥에서 덧붙이는 **것이 훨씬 효과적**.

실습 노트북 (17.2)

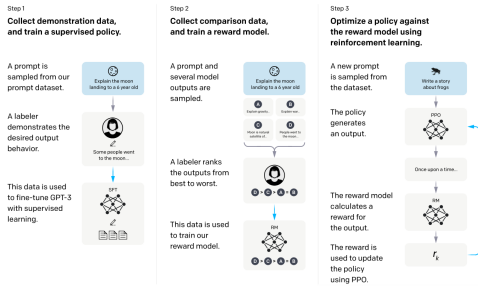
- 17.1의 next_token_distribution 구현과 **zero-shot vs few-shot** 출력 대조 (본문 코드 블록 재현).
- temperature를 붙인 장난감 모델 — 같은 “사과는” 프롬프트에서 $T = 1.0$ vs $T = 3.0$ 의 20회 샘플링 결과 비교(같은 프롬프트에서 왜 매번 다른 답이 나오는가).
- **temperature 곡선** — logits [2.0, 1.0, 0.5, 0.2]에 $T \in [0.2, 3.0]$ 적용 시 최상위 확률의 변화(본문 온도 그림).
- **CoT vs 직접 답 확률 시뮬레이션** — 3단계 추론, 부분 추론 성공률(중간 결과 조건으로 쓸 때 0.9 vs 없으면 0.5), 4,000회 반복: 직접 $\approx 0.5^3 = 0.125$, CoT $\approx 0.9^3 = 0.729$.

프롬프팅과 추론을 더 깊이: Stanford CS224N(Natural Language Processing with Deep Learning).

17.3 포스트트레이닝: SFT, RLHF, DPO

사전학습만으로는 부족한 이유: InstructGPT

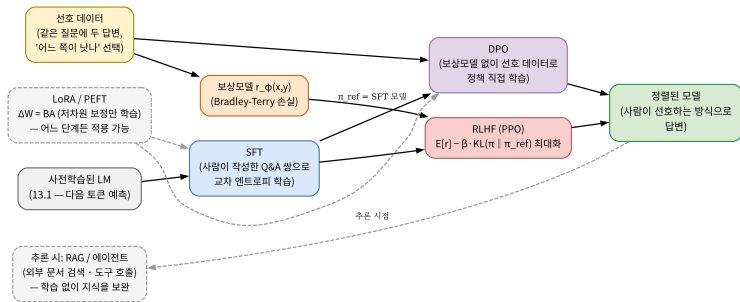
- 2022년 OpenAI **InstructGPT**(Ouyang et al., 2022) 놀라운 관찰: 파라미터가 **100배 더 적은** 모델이라도 사람이 원하는 방향으로 다듬어졌다면, 사전학습만 거친 거대 모델보다 사람들이 **훨씬 더 선호**.
- 17.1의 사전학습(다음 토큰 예측)은 “**그럴듯한 텍스트**”를 만들어내지만, 그게 “사람이 원하는 대답”이라는 보장은 아님.
- 사전학습 데이터에는 훌륭한 글도, 엉뚱하거나 무례한 글도 **섞여** 있기 때문.
- **포스트트레이닝**: 사전학습된 모델을 “사람이 실제로 원하는 방향”으로 다듬는 단계.



InstructGPT 3단계 학습 파이프라인(원 논문 Figure 2) — (1) SFT, (2) 보상모델(RM) 학습, (3) 이 보상모델을 이용한 PPO 강화학습.

17.3의 지도: 왜 정렬이 필요한가

- 정렬(alignment) = 데이터 분포 안에서 사람이 선호하는 쪽의 확률을 끌어올리는 **재조정(reweighting)**. LM 분포가 “그럴듯한 문장을 고르는 기준”이었다면, 정렬은 “사람이 고를 문장” 쪽으로 기준을 옮기는 것 — **새 능력을 넣는 게 아니라, 이미 있는 능력의 선택 기준을 바꾸는 것.**



- 사전학습된 LM이 **SFT**(사람이 작성한 Q&A)와 **선호 데이터**(두 답변 비교)를 거쳐 두 갈래: **보상모델(BT) 학습** → **RLHF(PPO)** 경유, 또는 **DPO**로 선호 데이터에서 정책을 직접 학습 — 모두 “정렬된 모델”로 수렴.

SFT: “이상적인 답변”을 그대로 흉내 내기

- **SFT**(Supervised Fine-Tuning): 가장 단순한 포스트트레이닝. 사람이 직접 작성한 “이상적인 (질문, 답변)” 쌍 $\{(x_i, y_i^*)\}$ 를 모아, 17.1의 다음 토큰 예측 손실 **그대로** — 다만 데이터가 인터넷 텍스트가 아니라 사람이 쓴 고품질 예시로.
- **답변 부분의 토큰에만** 교차 엔트로피 부과:

$$\mathcal{L}_{\text{SFT}}(\theta) = - \sum_i \sum_t \log \pi_{\theta}(y_{i,t}^* \mid x_i, y_{i,<t}^*)$$

- 변한 것은 **데이터뿐** — “질문을 잘 따라오는 이상적인 답변”.
- 17.2에서 “프롬프팅이 작동하려면 SFT가 미리 깔려 있어야 한다” — 그 “미리 깔려 있는 것”이 바로 이 단계의 산출물.

문제: 확장성 — “좋은 답변이 무엇인가”를 사람이 매번 처음부터 **작성**하는 것은 비용이 크다. 그리고 SFT 손실은 y^* 에 **확률 1**을 몰아주는 방향만: “사람이 쓰지 않았을 것 같은” 더 나은 답변을 탐색하는 압력이 **없다**. **모작(copy)**은 가르치되, **비판(비교)**은 가르치지 않는다 — 이 한계가 다음 단계의 동기.

손으로 한 번: SFT가 분포를 어떻게 바꿀까

- 어휘 7개 $\{A_1, \dots, A_7\}$, 7단계 이상적인 답변. 각 단계에서 정답 토큰에 주는 확률:
 - ▶ 사전학습(기준) 모델: (0.2, 0.5, 0.3, 0.4, 0.3, 0.5, 0.6)
 - ▶ SFT 이후 모델: (0.8, 0.7, 0.5, 0.6, 0.4, 0.6, 0.8)
- 사슬 분해 — 이상적 답변 전체 확률 = 각 단계의 곱:

$$P_{\text{ref}}(y^*) = 0.2 \times 0.5 \times 0.3 \times 0.4 \times 0.3 \times 0.5 \times 0.6 \approx 0.0011$$

$$P_{\text{SFT}}(y^*) = 0.8 \times 0.7 \times 0.5 \times 0.6 \times 0.4 \times 0.6 \times 0.8 \approx 0.0323$$

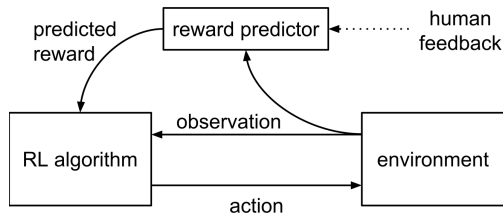
- **약 30배.** 토큰 하나당 손실(평균 $-\log p$) 로 보면 $0.98 \rightarrow 0.49$ — 둘 다 “잘 예측”하지만, SFT는 **정답 답변 전체의 합적 확률을 30배로 끌어올림.**

핵심: “각 단계는 0.2에서 0.8로만 올랐는데”

곱으로 쌓이므로 짧은 답변에서도 큰 격차 — 17.1의 사슬 분해가 포스트트레이닝에서 다시 작동하는 모습.
(정확한 값은 실습 노트북 Section 1에서 검증.)

RLHF: 작성 대신 비교로

- **RLHF**(Reinforcement Learning from Human Feedback, Christiano et al., 2017): 다른 전략.
- 사람이 매번 “정답”을 새로 쓰는 대신, 모델이 만든 **여러 답변 중 어느 쪽이 더 나은지 고르기만** 하면 됨 — 비교가 작성보다 훨씬 쉽고 빠름.
- 이 선호 데이터로 **보상모델** (reward model)을 학습.
- 그 보상모델의 점수를 **최대화**하도록 언어모델 자체를 조정.



RLHF 구조(원 논문 Figure 1) — 보상 예측기(reward predictor)가 트래젝터리 세그먼트의 인간 비교 피드백으로 비동기 학습된 뒤, 그 보상을 이용해 정책을 강화학습으로 학습하는 파이프라인.

1단계 — 보상모델: Bradley-Terry 모델

- 같은 질문 x 에 대해 모델이 만든 두 답변 y_w (**winner**, 더 선호)와 y_l (**loser**, 덜 선호). 보상모델 $r_\phi(x, y)$ 가 y_w 에 더 높은 점수를 주도록 학습.
- **브래들리-테리**(Bradley-Terry) 모델 — 선호 확률을 시그모이드로:

$$P(y_w \succ y_l \mid x) = \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))$$

- 이 확률을 최대화하는 손실 = **Ch. 2 로지스틱회귀와 같은 교차 엔트로피 형태** — “정답 라벨”이 “ y_w 가 이겼다”는 사실 하나뿐인 이진분류이므로.

왜 시그모이드인가 (Ch. 2와 같은 이유)

(1) 점수 차이 $\in \mathbb{R}$ 을 $[0,1]$ 확률로 매핑, (2) 점수 차이가 선호 확률의 **로그-odds**가 선형 스케일로: 차이 +1 $\rightarrow$ odds e 배, +2 $\rightarrow e^2$ 배 (17.1 softmax의 “원점수 차이 = 확률 비율”과 동일 구조), (3) 극단적 차이에서도 손실이 0에 **유한하게** 접근($-\log \sigma(z) \sim e^{-z}$) $\rightarrow$ 잘 구분하는 쌍이 학습을 지배하지 않음.

BT 손실: 코드와 숫자

```
import math

def sigmoid(z):
    return 1 / (1 + math.exp(-z))

def reward_model_loss(r_win, r_lose):
    # Bradley-Terry:  $P(y_w > y_l) = \text{sigmoid}(r_{\text{win}} - r_{\text{lose}})$ 
    return -math.log(sigmoid(r_win - r_lose))
```

- **손으로:** $r_w = 1, r_l = -1$ (margin $z = 2$) $\Rightarrow \mathcal{L} = -\log \sigma(2) \approx -\log 0.8808 \approx \mathbf{0.1269}$.
- **거꾸로** ($r_w = -1, r_l = 1$): $-\log \sigma(-2) \approx \mathbf{2.1269}$ — 잘 구분한 쌍보다 **약 17배** 큼.
- **margin 0**(무작위 추측): $\log 2 \approx 0.693$.
- **로그-odds로 읽기:** $z = 2$ 일 때 odds는 $e^2 \approx 7.4$ — “이 사람이 y_w 를 y_l 보다 선호할 odds가 약 7:1”.

손으로: 2차원 보상모델을 실제로 학습

- BT 손실은 로지스틱회귀와 같은 모양 $\rightarrow$ 로지스틱회귀의 기계(구현, SGD)를 그대로 재사용. 장난감 보상모델 $r_\phi(x, y) = w_1 \cdot \text{길이} + w_2 \cdot \text{거절 여부}$.

질문	winner (길이, 거절)	loser (길이, 거절)
Q1	(2, 0)	(5, 0)
Q2	(3, 0)	(7, 1)
Q3	(4, 0)	(8, 2)
Q4	(2, 0)	(3, 1)
Q5	(5, 0)	(4, 1)

- winner는 모두 더 **짧고** 거절하지 않는 쪽. $w = (0, 0)$ 에서: 모든 margin 0 $\rightarrow$ 손실 $5 \times \log 2 \approx 3.47$, 5쌍 모두 50% “동점”. $\eta = 0.5$ 로 몇 백 번 $\rightarrow w \approx (-1.90, -6.69)$, 5쌍 모두 정확, 평균 손실 ≈ 0.002 (노트북 Section 2).

(1) 두 특징 모두 부호가 음수 — “짧을수록, 거절하지 않을수록 높게”의 방향을 모델이 스스로 찾음. (2) 스케일이 다르면 크기가 다름 — 거절 ± 1 , 길이 $\pm 3 \sim 4 \Rightarrow w_2$ 가 w_1 의 약 3.5배. (3) 5쌍은 선형분리 가능이라 완전 분리 가능 — 실제 (질문, 답변) 쌍은 훨씬 고차원이고 분리 불가가 많아, **margin의 순서**만 맞으면 되고 “손실이 안 떨어진다”는 사실 자체가 선호 데이터의 모순을 알려주는 신호.

보상모델은 “점수”가 아니라 “상대 순서”

- 위 장난감 실험이 보여주는 것: 보상모델이 “점수를 만드는 것”이 아니라 “상대 비교”의 일관된 순서를 만드는 것.
- r_ϕ 에 상수를 더해도(모든 점수 +100) 모든 margin이 불변 $\rightarrow$ BT 손실도 불변. $\Rightarrow$ 점수는 절대값에 의미가 없고 차이(margin)에만 의미가 있다.
- 2단계 — 강화학습으로 정책 학습: LM을 “질문 x 라는 상태에서, 답변 y 라는 행동을 토큰 하나씩 순차적으로 고르는 정책”으로 취급.
- 보상모델 점수 $r_\phi(x, y)$ 를 보상 삼아 강화학습(상태·행동·보상; 이번 학기 주제 아님, ML2 전문)으로 파라미터 조정.

왜 “토큰 하나씩”이 강화학습인가

목표 함수의 관점에서 전혀 다른 문제: SFT는 정답 토큰이 주어지므로 각 토큰 손실을 독립으로 계산 가능. RLHF는 정답이 없다 — 보상모델은 완성된 답변 전체에 하나의 점수만 줄 뿐. “높게 점수 받은 공로가 어느 토큰에 있는가” (credit assignment)를 가르치는 것이 강화학습이 존재하는 이유.

17.2 CoT와 대비: CoT는 중간 결과를 출력물에 써서 다음 토큰 예측의 조건으로(추론 시점), RLHF는 출력 전체의 보상을 출력 경로의 확률 (로그-우도)에 비례해 역분배(학습 시점). “프롬프팅은 즉각적이고 파인튜닝은 영구적”의 46/1

PPO: 목적함수의 형태

- 이때 가장 널리 쓰이는 알고리즘: **PPO**(Proximal Policy Optimization, Schulman et al., 2017). 유도는 ML2에서 — 여기서는 목적함수의 **형태만**:

$$J(\theta) = \mathbb{E}[r_\phi(x, y)] - \beta D_{KL}(\pi_\theta(\cdot|x) \parallel \pi_{\text{ref}}(\cdot|x))$$

- 앞항: 보상모델 점수의 기대를 최대화. 뒤항: **기준 모델**(사전학습 + SFT 후의 π_{ref})에서 벗어나는 정도에 β 로 **벌**.
- **KL 수치의 감각**: 2.3절의 $D_{KL}(p||q) = \sum p \log(p/q)$. 두 토큰 어휘 $\{A, B\}$, $\pi_{\text{ref}} = (0.5, 0.5)$ 라 하면:
 - ▶ 정책이 기준과 **정확히 같으면** KL = 정확히 0(동일 분포, 이탈 없음).
 - ▶ A에 몰아붙인 극단 $\pi = (0.95, 0.05)$:

$$D_{KL} = 0.95 \cdot \log(0.95/0.5) + 0.05 \cdot \log(0.05/0.5) \approx 0.610 - 0.115 \approx 0.495$$

$\beta = 1$ 이면 이 이탈에 0.5의 **벌**.

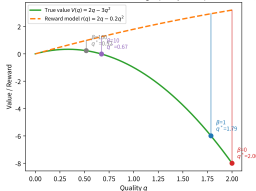
- KL이 0(동일)에서 0.5(극단적 편향)으로 커지는 모습 = “**기준 모델의 말투에서 얼마나 벗어났는가**”를 단일 숫자로 재는 **이탈 거리**.
- PPO는 LLM 정렬을 위해 발명된 것이 **아니다**. 2017년 OpenAI가 발표한 PPO는 **로봇 제어**(인공신경망 정책 + 연속 행동)에서 “정책 업데이트를 **한 번에 크게 하지 마라**”는 안전 장치를 **클리핑**(clipping)이라는 단순한 수단으로 구현.

보상 해킹을 숫자로: KL 항 없으면

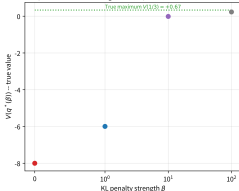
- KL 항을 없애면($\beta = 0$) 최적화는 “보상모델 점수 $r(q)$ 를 그냥 **최대화**”로 단순화.
- 1차원 장난감: 품질 $q \in [0, 2]$, **진짜 가치** $V(q) = 2q - 3q^2$, **보상** $r(q) = 2q - 0.2q^2$. 둘은 저품질($q \lesssim 0.6$)에서 거의 일치하지만, 고품질 구간으로 갈수록 보상모델은 **실제 가치의 하락을 과소평가** — 학습 데이터가 “보통 품질”에 집중 $\rightarrow$ **외삽 오차**.

β	q^*	보상 $r(q^*)$	KL	진짜 가치 $V(q^*)$
0	2.00	3.20	1.125	-8.00
1	1.79	2.93	0.827	-5.99
10	0.67	1.26	0.015	-0.01
100	0.52	0.98	0.0002	+0.23
(V 진짜 최대)	0.33	—	—	+0.67

Reward model underestimates V at high quality (extrapolation error)



$\beta=0$: reward hacking ($V=-8$) / $\beta=100$: frozen ($V=+0.23$)



$\beta = 0$: $q^* = 2.00$ 까지 밀려나 $V = -8.00$ /
 $\beta = 100$: $q^* = 0.52$ 동결 ($V = +0.23$) / 진짜 최대
 $q = 1/3$ ($V = +0.67$).

- $\beta = 0$: **경계** $q = 2$ 까지 밀려 보상 점수 3.20을 얻지만 **진짜 가치는 -8** — 보상 점수는 계속 오르는 데 실제 품질은 무참히 추락. 전형적 **보상 해킹**(Gao et al., 2022).

- $\beta = 10$: $q^* \approx 0.67$, $KL \approx 0.015$ 거의 0 — 하지만 V 의 진짜 최대($q = 0.33$, $+0.67$)는 여전히 놓침.

DPO: 보상모델도 RL 루프도 없이

- **DPO**(Direct Preference Optimization, Rafailov et al., 2023): RLHF는 (1) 보상모델을 따로 학습하고 (2) 그 보상모델로 강화학습을 돌리는 2단계 — 복잡하고 불안정할 수 있음.
- DPO가 수학적으로 보이는 것: RLHF 목적함수의 **최적 정책**이 보상함수를 π_θ 와 π_{ref} 의 **비율**로 직접 표현할 수 있음을.
- 그 관계를 보상모델 손실에 대입 → **보상모델도 강화학습 루프도 없이, 선호 데이터에서 정책을 곧바로 학습하는 손실:**

$$\mathcal{L}_{\text{DPO}} = -\log \sigma \left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right)$$

핵심

형태를 보면 여전히 위의 보상모델 손실과 **똑같은 로지스틱 손실** — 다만 “보상”의 역할을 $\beta \log(\pi_\theta/\pi_{\text{ref}})$ (정책이 기준 모델보다 그 답변을 얼마나 더 선호하게 됐는가)가 대신한다. **RLHF의 2단계 과정 전체를, 지도학습 손실 하나로 대체.**

DPO의 3단계 논리: 왜 π_{ref} 의 비율인가

- RLHF 목적함수 $J(\theta) = \mathbb{E}[r] - \beta D_{KL}$ 을 π_{θ} 에 대해 최적화하면 (토큰 위치마다 독립 $\rightarrow$ 인수분해), 최적 정책은

$$\pi^*(y|x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y|x) e^{r(x,y)/\beta}$$

— 닫힌 형태(closed form). $Z(x)$ = 질문 x 에 대한 노멀라이제이션 상수.

- 다시 말해 보상함수 r 자체가

$$r(x, y) = \beta \log \frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z(x)$$

로 역산(inversion) — “최적 정책과 보상함수는 서로의 변환”.

- 이 r 을 BT 손실에 대입하면, $\beta \log Z(x)$ 가 y_w, y_l **둘에 공통**으로 들어와 차이를 취할 때 **소거** — $Z(x)$ 를 계산할 **필요 없이**, π_{θ} 와 π_{ref} 의 **비율**만으로 BT 손실이 쓰여진다.

이 소거가 DPO의 핵심 한 줄: “보상함수를 분리해서 학습(RLHF)”과 “보상함수를 정책 자체에 **내장**시켜 학습(DPO)”은 같은 BT 손실을 최적화하지만, 전자는 **보상모델**이라는 별도 파라미터를 두고, 후자는 **정책 파라미터**가 보상의 역할을 겸임.

- DPO 유도에서 **보상함수가 정책의 로그-비율로 표현**된다는 것은, “RLHF에서 최적화된 정책 = DPO에서 최적화된 정책”이 **같은 BT 손실을 최적화**한다는 것에 불과 — **동일한 데이터**,

숫자로: DPO 손실의 그래디언트 방향

- 17.2 장난감 상황(두 후보 $y_w, y_l, \beta = 1$). 초기: $\pi_\theta(y_w|x) = 0.6, \pi_\theta(y_l|x) = 0.2, \pi_{\text{ref}}(y_w|x) = 0.4, \pi_{\text{ref}}(y_l|x) = 0.3$.
- 두 로그-비율: $\log \frac{0.6}{0.4} \approx 0.4055, \log \frac{0.2}{0.3} \approx -0.4055$.
- margin $z = 0.4055 - (-0.4055) \approx 0.811$, 손실 $\mathcal{L} = -\log \sigma(0.811) \approx \mathbf{0.368}$.
- BT 그래디언트를 그대로: $\partial \mathcal{L} / \partial \log \frac{\pi_\theta(y_w)}{\pi_{\text{ref}}(y_w)} = \sigma(z) - 1 \approx -0.308$,
 $\partial \mathcal{L} / \partial \log \frac{\pi_\theta(y_l)}{\pi_{\text{ref}}(y_l)} = 1 - \sigma(z) \approx +0.308$ — y_w 의 로그-비율을 올리고 y_l 의를 내리는 방향이 정확히 BT 손실을 줄이는 방향.
- 학습 진행 $\pi_\theta(y_w|x) = 0.7, \pi_\theta(y_l|x) = 0.1 \Rightarrow z \approx 1.658$, 손실 ≈ 0.174 — 같은 BT 손실 곡선을, 별도 보상모델 없이 정책 확률로 직접 내림.
- $\beta = 0.1$ 이면 margin도 $1/10$ ($z \approx 0.081$), 손실 0.653 — β 는 “선호를 얼마나 강하게 반영할 것인가”의 강도 조절, RLHF의 KL 항 강도와 같은 역할.

LoRA: W 는 고정된 채 $\Delta W = BA$ 만 학습

- **LoRA**(Low-Rank Adaptation, Hu et al., 2022) / **PEFT**(Parameter-Efficient Fine-Tuning): 수십~수백억 파라미터 전체 파인튜닝은 계산·저장 비용이 큼.
- 원래 가중치 W 를 **고정된 채**, 훨씬 작은 두 저차원 행렬의 곱 $\Delta W = BA$ (B, A 의 차원이 W 보다 훨씬 작음)만 학습 $\rightarrow W + \Delta W$ 를 새 가중치로.
- 학습 파라미터 수를 **수백 분의 1**로 줄이면서도 실전에서 전체 파인튜닝과 **비슷한 성능**.
- Ch. 10.3 전이학습(“합성곱 층 고정, 분류층만 학습”)과 같은 정신. 차이: “일부 층을 통째로 고정” 대신 **모든 층에 아주 작은 저차원 보정**을 추가로 학습.
- **왜 “저차원”이 충분한가**: LoRA의 가설은 파인튜닝의 **효과적 변화** ΔW 가 본래 W 의 전체 차원보다 **낮은 계수(rank)**를 가진다는 것. $W \in \mathbb{R}^{20 \times 20}$ 에 rank-4 ΔW 를 SVD로 근사:

rank r	파라미터 수 (vs 전체 400)	상대 오차
4 (정확)	160 (40%)	≈ 0
3	120 (30%)	0.326
1	40 (10%)	0.701

rank-4를 rank-4로 표현 $\rightarrow$ 오차 0(정확 재현), 파라미터 40%로 축소. rank-3: 33%, rank-1: 70% — **진짜 rank를 넘는 순간 정보 손실**. 실제 LLM(4096×4096 한 층, 약 1.7×10^7 파라미터)에서 $r = 8$: $2 \times 4096 \times 8 = 65,536$ — 전체의 **0.39%**에 불과.

에이전트와 RAG: “모델이 필요할 때 바깥에 접근”

- 프롬프팅만으로는 **모르는 최신 정보**나 외부 문서를 답할 수 없다.
- **RAG**(Retrieval-Augmented Generation, Lewis et al., 2020): 질문과 관련된 문서를 **먼저 검색해** 프롬프트에 함께 넣어줌.
- **에이전트**(agent, ReAct: Yao et al., 2022): 한 걸음 더 — 모델이 검색·계산기·코드 실행 같은 **도구를 스스로 호출**하도록 만들.
- 둘 다 “파라미터 안에 모든 지식을 우겨넣는” 대신 “**모델이 필요할 때 바깥 정보/도구에 접근**하게 한다”는 **같은 방향**의 해법.

RAG를 17.2의 언어로

RAG는 **프롬프팅의 확장**: few-shot 예시가 “모델이 **조건으로** 삼는 토큰”을 확장한다면, RAG는 **검색된 문서를 그 조건으로 삼는 것**. 모델의 파라미터는 **아무것도 바뀌지 않고**, **컨텍스트에 문서를 넣는 행위 자체가 학습을 대체**.

예: “8.3 / 5.2 = ?”에 RAG가 **계산기 도구**를 호출해 1.596을 얻고, “그 값을 3으로 곱해줘”는 4.788을 산출. 이 과정은 17.2 환각 완화 수단(도구를 **프롬프트 바깥에서** 덧붙인다)의 **자동화** 버전. “스스로 판단”이 **에이전트**의 핵심 — “다음 토큰이 **도구 호출**이라는 특수 토큰”일 가능성을 모델이 학습한 것.

세 방식 비교: SFT vs RLHF vs DPO

	SFT	RLHF	DPO
사람이 주는 데이터	“이 답변이 좋다”(작성)	“이쪽이 더 낫다”(비교)	“이쪽이 더 낫다”(비교)
학습 대상	정책 π_θ	보상모델 r_ϕ + 정책 π_θ	정책 π_θ
손실	다음 토큰 예측 CE	BT(RM) + PPO(RL)	BT(정책 로그-비율)
보상모델	없음	별도 학습	없음(정책이 겸임)
강화학습 루프	없음	있음(불안정, 보상 해킹 위험)	없음
KL 항의 역할	—	보상 해킹 방지(β 튜닝)	β 가 같은 역할
상한	사람이 작성할 수 있는 품질	(보상모델의 근사 한계)	(데이터의 선호 순서 한계)
LoRA 적용	가능	가능(RM도, 정책도)	가능

계층도 구분할 것

LoRA = 학습 단계의 효율화(어느 파인튜닝 단계든 적용 가능) / **RAG/에이전트** = 추론 단계의 지식 보완(학습 없이 컨텍스트에 넣음) — 둘을 동시에 쓰는 것이 실무 표준.

“LoRA 파인튜닝 + RAG 최신 정보 공급 + 에이전트 도구 호출”은 각각 **서로 다른 단계**의 문제(가중치 비용 / 지식 시효 / 계산 능력)를 해결하는 **독립 축**.

17.3 자주 하는 실수

- ❶ 실수: “RLHF는 사람 선호를 모델에 직접 주입하는 것 아닌가?” **아님**. 선호 데이터는 **보상모델**의 학습에 쓰이며, 정책은 보상모델의 점수(간접 신호)를 따라간다. 사람이 직접 정책을 가르치는 것은 SFT.
- ❷ 실수: “KL 항은 불필요한 복잡함 아닌가?” **아님**. KL 항(또는 DPO의 β)은 **보상 해킹**과 **과소최적화**의 균형을 조율하는 **핵심** 하이퍼파라미터 — 없으면 정책이 보상모델의 **외삽 오차**를 노리거나, 너무 강하면 아무것도 안 학습.
- ❸ 실수: “DPO는 무조건 RLHF보다 낫다?” **아님**. DPO는 **2단계를 1단계로** 압축한 것 — 보상모델을 **별도 평가 기준**으로 재사용 하거나, 보상모델을 **여러 개로** 학습해 평균 내는 **유연성**은 RLHF만 가짐.
- ❹ 실수: “LoRA rank가 클수록 좋다?” **아님**. rank $\rightarrow$ **과적합**과 **파라미터 폭증**. 실제로는 **작은 rank**(4 32) 가 대부분 충분 — **효과적 변화**가 낮은 계수라는 **가설**을 반영.
- ❺ 실수: “SFT 데이터가 많을수록 좋다?” **아님**. 데이터 **품질**(다양성, 정확성)이 수량보다 중요 — InstructGPT는 **약 1만 3천 개**의 예시로도 **GPT-3 175B**를 뛰어넘.

17.3 연습문제

- 1 DPO 손실 $\mathcal{L} = -\log \sigma(\beta[\log \frac{\pi_{\theta}(y_w)}{\pi_{\text{ref}}(y_w)} - \log \frac{\pi_{\theta}(y_l)}{\pi_{\text{ref}}(y_l)}])$ 에서 $\beta = 1$, $\pi_{\theta}(y_w) = 0.7$, $\pi_{\text{ref}}(y_w) = 0.4$, $\pi_{\theta}(y_l) = 0.1$, $\pi_{\text{ref}}(y_l) = 0.3$. margin z 와 손실값을 계산하고, β 가 0.1이면 z 가 어떻게 되는가?
- 2 PPO 목적함수 $J(\theta) = \mathbb{E}[r(x, y)] - \beta D_{KL}$: (a) $\beta = 0$ 이면 최적해가 어디로 흐르는가? (b) $\beta \rightarrow \infty$ 이면? (c) 17.3의 1차원 장난감에서 $\beta = 100$ 이면 $q^* \approx 0.67$, $V(q^*) \approx -0.01$ 인데 — **진짜** **가치의 최대** ($q = 0.33$, $V = +0.67$) 를 놓치는 이유를 보상모델의 **외삽 오차**로 설명.
- 3 LoRA: $W \in \mathbb{R}^{4096 \times 4096}$ 에 $r = 8$ 이면 학습 파라미터가 몇 개인가? 전체 1.68×10^7 대비 비율은?
- 4 **판별**: “RLHF에서 KL 항이 클수록 보상 해킹이 더 잘 막힌다” — 참/거짓, 이유.
- 5 **판별**: “DPO는 보상모델을 학습하지 않기 때문에 보상 해킹이 불가능하다” — 참/거짓, 이유 (힌트: DPO의 β 와 KL 항의 관계).
- 6 **판별**: “RAG는 모델의 지식을 향상시킨다” — 참/거짓, 이유 (17.2 “컨텍스트 = 조건”과의 관계).

예제: DPO 손실(장난감)

```
import numpy as np
from scipy.special import expit # logistic(sigmoid)

def dpo_loss(pi_th_w, pi_ref_w, pi_th_l, pi_ref_l, beta=1.0):
    margin = beta * (np.log(pi_th_w/pi_ref_w) - np.log(pi_th_l/pi_ref_l))
    return -np.log(expit(margin))
```

17.3 확인 문제

- 1 SFT 손실 $\mathcal{L} = -\frac{1}{T} \sum \log \pi_{\theta}(y_t | y_{<t})$ 에서, SFT의 **입력 데이터**는 (질문, 정답 답변) 쌍 — 이 “정답 답변”을 누가 작성하는가?
- 2 RLHF 2단계에서 **보상모델**이 학습받는 손식은? **정책**이 학습받는 목적함수(17.3의 $J(\theta)$)를 쓰고, KL 항이 **왜** 필요한가?
- 3 DPO 손실을 BT 손실과 **비교**: BT의 $\log r(x, y_w) - \log r(x, y_l)$ 대신 들어가는 항을 쓰고, β 가 어떤 역할을 하는가.
- 4 **판별**: “DPO는 보상모델이 필요 없으므로 RLHF보다 항상 안정적이다” — 참/거짓, 이유(보상 해킹 관점).
- 5 LoRA rank $r = 8$ 을 $r = 64$ 로 올리면: (a) 학습 파라미터 수가 몇 배? (b) **과적합** 위험은 어떻게 변하는가? (c) 17.3의 SVD 근사 결과(rank-4: 0.0%, rank-3: 0.3%, rank-1: 0.7%) 와 비교.
- 6 RAG: 질문과 **검색된 문서**를 컨텍스트에 넣는다 — 17.2의 “컨텍스트 = **조건**” 관점에서 RAG는 학습인가, **조건화**인가?